

Spring Boot Starter

빌드 시스템, 의존성 관리, starter, 관리되는 의존성 버전

작성 기준: 원문 페이지를 확인한 뒤 만든 학습용 한국어 번역 요약본입니다. 공식문서 전체 문장을 그대로 복제한 전문 직역본은 아니며, 원문 URL을 함께 남겼습니다.

[1] 전체 구조 설명

- Spring Boot 애플리케이션은 보통 Maven 또는 Gradle 같은 빌드 시스템으로 시작한다.
- 빌드 시스템은 라이브러리를 가져오고, Spring Boot는 검증된 의존성 조합을 관리한다.
- Starter는 자주 함께 쓰는 의존성을 하나의 묶음으로 제공해서 프로젝트 초기 설정을 줄인다.
- Dependency Versions 부록은 Spring Boot가 어떤 라이브러리 버전을 관리하는지 확인하는 색인 역할을 한다.

[2] 문서별 직역체 학습 정리

Build Systems / Starters

- 공식문서는 의존성 관리가 되는 빌드 시스템을 쓰라고 안내한다. 이유는 Spring Boot가 지원하는 라이브러리 조합을 일관되게 관리하기 때문이다.
- Spring Boot 릴리스마다 지원되는 의존성 목록이 정리되어 있으며, 보통 개발자가 각 라이브러리 버전을 직접 쓰지 않아도 된다.
- Starter는 편리한 의존성 설명자다. 예를 들어 JPA 데이터 접근을 시작하려면 여러 의존성을 직접 조합하지 않고 spring-boot-starter-data-jpa를 추가한다.
- 공식 starter 이름은 spring-boot-starter-* 패턴을 따른다. 서드파티 starter는 공식 artifact와 충돌하지 않게 spring-boot로 시작하지 않는 것이 원칙이다.
- Actuator, Web, Security, Validation처럼 기능 단위 starter가 있고, Tomcat/Jetty/Log4j2 같은 기술 facet을 바꾸는 starter도 있다.

Dependency Versions

- Dependency Versions 문서는 Spring Boot가 관리하는 의존성의 상세 목록이다.
- 이 부록은 어떤 라이브러리가 Spring Boot BOM에 의해 관리되는지 확인할 때 사용한다.
- Spring Boot 버전을 올리면 관리되는 의존성도 함께 일관되게 올라간다. 그래서 버전 충돌을 줄일 수 있다.
- 직접 버전을 지정해 Spring Boot 권장값을 덮어쓸 수는 있지만, 특별한 이유가 없으면 관리 버전을 쓰는 편이 안정적이다.

[3] 핵심 실행 흐름

프로젝트 생성 → Maven/Gradle 선택 → 필요한 starter 추가 → Spring Boot BOM이 transitive dependency 버전 관리 → 애플리케이션 실행과 테스트

[4] 중요한 포인트 정리

- Starter는 실제 코드를 실행하는 클래스가 아니라 의존성 묶음이다.

- BOM은 라이브러리 버전 표준표라고 이해하면 된다.
- starter를 많이 넣는다고 좋은 것이 아니다. 필요한 기능 단위로만 추가해야 의존성 표면이 작다.

원문 URL

<https://docs.spring.io/spring-boot/reference/using/build-systems.html>

<https://docs.spring.io/spring-boot/appendix/dependency-versions/index.html>